



COPPE/UFRJ Programa de Engenharia Civil

Algoritmos Genéticos COC 769

Beatriz de S. Leite Pires de Lima
Solange Guimarães

bia@coc.ufrj.br
sol@coc.ufrj.br

Programa

- ◆ Introdução a Computação Evolutiva
- ◆ Otimização, algoritmos de busca
- ◆ Estratégias Evolutivas
- ◆ Algoritmos Genéticos
- ◆ Aspectos Avançados
- ◆ Problemas de Permutação, Aprendizagem de máquina
- ◆ Aplicações

O que é Otimização?

Problemas de otimização aparecem em várias áreas da ciência, engenharia, indústria, negócios.

Tomar decisão: **o que é melhor?**

Problemas do dia a dia: escolha um plano de telefone ou agendamento de aulas numa instituição de ensino

Qualquer problema que envolva projeto (*design*) normalmente apresenta como **mérito: custo** ou **eficiência**.

deseja-se reduzir custo e manter um nível aceitável de qualidade/desempenho

Num espaço de soluções possíveis deseja-se encontrar uma solução que seja a que **melhor** alcança um determinado **objetivo**.

Métodos de busca

Um algoritmo de busca terá como entrada um problema e retornará como saída uma solução.

Uma ou mais soluções candidatas podem ser utilizadas no processo de busca.

Os métodos que utilizam mais de uma solução (indivíduo) no processo de busca são denominados de métodos populacionais como por ex., os algoritmos evolutivos.

O primeiro passo na resolução de um problema é a formulação do problema, que irá depender das informações disponíveis

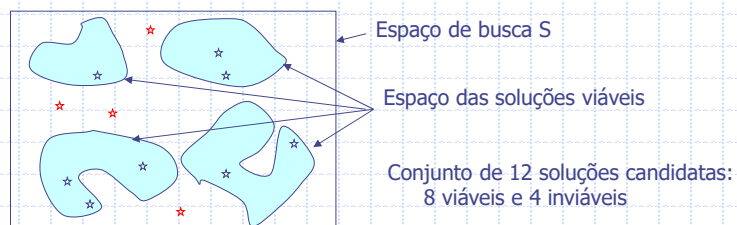
Nomenclatura

A formulação do problema consiste :

- ◆ **Função objetivo:** Descrição de uma meta. Trata-se de uma expressão matemática que descreve o problema (representa a quantidade a ser otimizada)
- ◆ **Variáveis de projeto/problema:** São aquelas para as quais o problema é resolvido. São as variáveis que afetam o valor da função objetivo.
- ◆ **Conjunto de restrições:** restringem os valores que podem ser atribuídos às variáveis do problema. Funções de igualdade ou desigualdade descrevem os limites desejáveis do problema.

Nomenclatura

- ◆ **Espaço de busca:** Espaço onde estão todas as possíveis soluções do problema

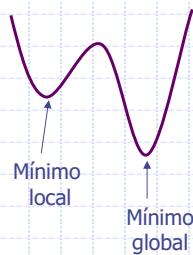


Otimização

O conceito de **ótimo global**: mínimo ou máximo global para qualquer ponto viável.

◆ Problema unimodal ou multimodal

- Função objetivo tem um ou mais pontos de mínimo ou máximo:



Achar o mínimo global não é tarefa fácil para problemas complexos.

Problemas de otimização

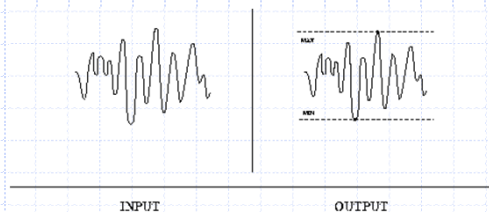
Otimização de uma função simples

Consiste em encontrar um vetor $x \in M$

M - região de soluções viáveis,

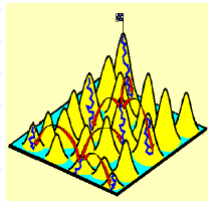
tal que: a função f seja maximizada (ou minimizada):

Maximizar $f(x)$



Problemas de otimização

A função f pode ser complexa:



- Multimodalidade;
- Dimensionalidade;
- Fortes não-linearidades;
- Não-diferenciabilidades;
- ruídos;
- ótimos variantes no tempo;
- etc.

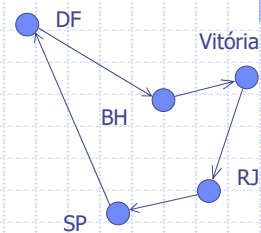
Estas funções complexas levam a problemas de otimização de difícil solução, ou algumas vezes, insolúveis.

Ex. problemas de otimização

Problema do Caixeiro Viajante – PCV (Travelling Salesman Problem – TSP)

Um vendedor precisa visitar uma lista de cidades em sua área passando por cada uma somente uma vez, e depois retornar ao ponto de origem.

Dado o custo da viagem entre as cidades, qual o itinerário de menor custo?



PCV - problema de otimização combinatória; aparece em muitas aplicações e uma classe enorme de problemas podem ser reduzidos a ele.

Problema complexo: tem sido resolvido por algoritmos heurísticos, não garantem achar a solução exata, mas reduzem o tempo computacional.

Métodos Clássicos de Otimização

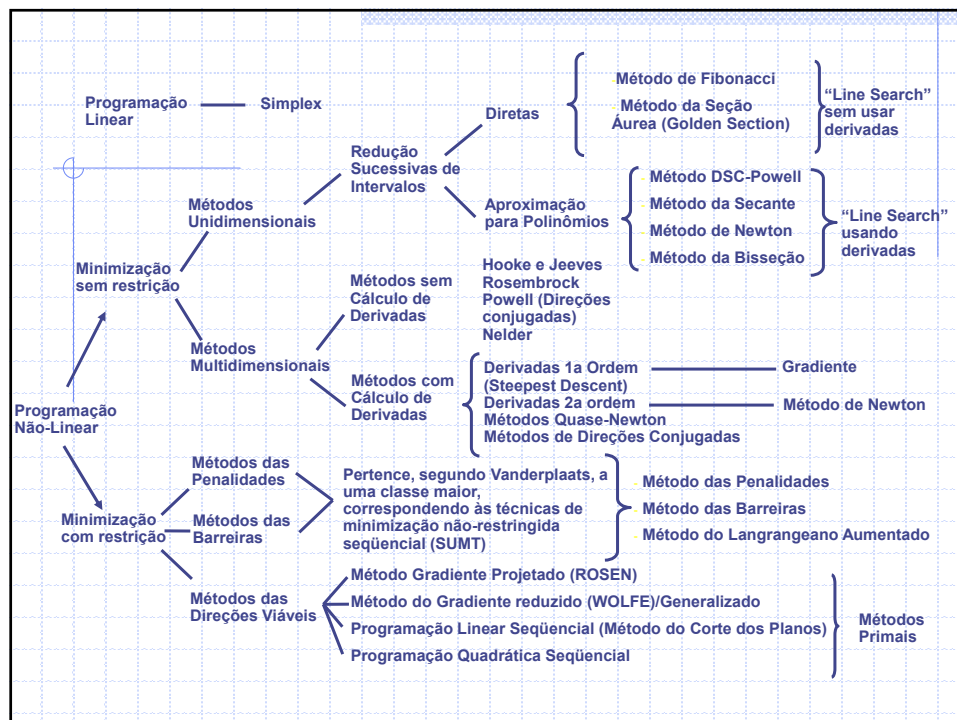
Algoritmos clássicos eficientes de otimização:
usualmente empregam **derivadas** e convergem para **ótimo local**.

Para problemas com restrições:

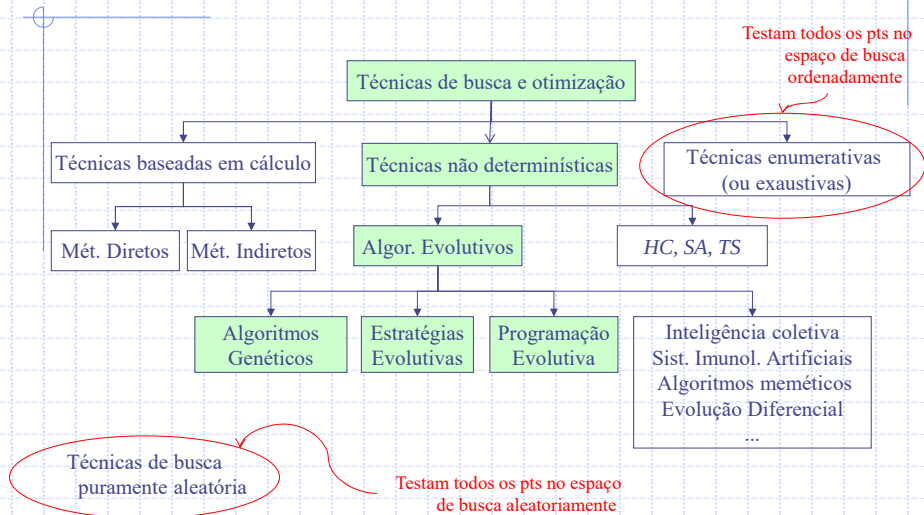
achar pontos que satisfaçam todas as restrições é normalmente uma tarefa difícil → **penalidades**

Em geral:

- Programação linear: Quando tanto as restrições como a função objetivo são funções lineares das variáveis de projeto;
- Programação não-linear: Quando a função objetivo ou pelo menos uma das restrições é função não-linear das variáveis de projeto



Resumo das técnicas de busca e otimização



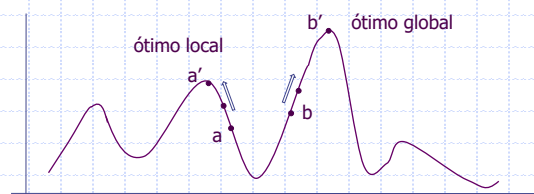
Algumas técnicas de Busca Local

- Hill Climbing : Método de busca que utiliza uma técnica de melhora iterativa
 - Hill Climbing simples
 - Hill Climbing iterativo
 - Hill Climbing estocástico
- Metaheurísticas baseadas em busca local:
 - Simulated Annealing: explora uma possível analogia com a termodinâmica
 - Tabu Search: uso de uma memória flexível para tornar o processo de busca mais eficaz

Hill Climbing

Algoritmo Simples:

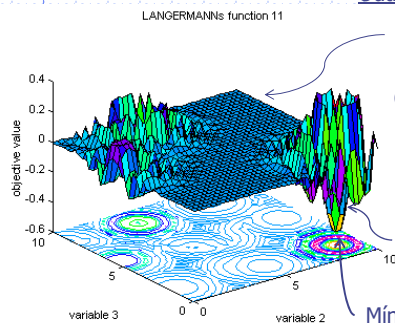
1. É aplicado a um único ponto (solução candidata) no espaço de busca
2. A cada iteração, um novo ponto na vizinhança do ponto atual é selecionado
3. Se este novo ponto apresenta um melhor valor para a função de avaliação, então o novo ponto torna-se o ponto atual
4. O método termina quando nenhuma melhora é alcançada ou um número fixo de iterações foi efetuado



Hill Climbing

- O algoritmo simples realiza uma busca local em torno de seu ponto inicial.
- O sucesso de uma simples iteração depende do ponto inicial escolhido.
- Converge para ótimos locais.

Outros problemas: platôs e cumes



Platôs - função de avaliação dá o mesmo resultado, algoritmo se perde

Cume - algoritmo encontra sucessivos ótimos locais e converge lentamente

Vários ótimos locais: difícil localizar o ótimo global

Mínimo global



Hill Climbing Iterativo- *Iterated hill climbing*

- Várias execuções do algoritmo, reiniciando a partir de diferentes posições
- Armazena a melhor solução obtida

Algoritmo:

Inicializa x_{Best} (melhor solução encontrada até o momento)

Para cada x_a , faça

Sempre
retorna
ótimo
local

1. Selecione um novo ponto (x_n) na vizinhança do ponto atual (x_a)
2. Se x_n melhor que x_a então $x_a \leftarrow x_n$
3. Termina quando nenhuma melhora é alcançada ou um número fixo de iterações foi efetuado

Se x_a melhor que x_{Best} então $x_{Best} \leftarrow x_a$

Escapa do ótimo local
começando nova
busca até no. Max de
tentativas

Hill Climbing Estocástico

- Um novo ponto na vizinhança é selecionado probabilisticamente (ao invés de testar todos os pontos da vizinhança)
- Esta probabilidade depende da diferença entre o ponto atual e a solução selecionada

Algoritmo:

1. Selecione um ponto x_a aleatoriamente
2. Selecione um novo ponto (x_n) na vizinhança do ponto atual (x_a)
3. Selecione x_n com probabilidade $p = \frac{1}{1 + e^{\frac{eval(x_a) - eval(x_n)}{T}}}$
4. Termina quando nenhuma melhora é alcançada ou um número fixo de iterações foi efetuado

Parâmetro T é mantido constante durante a execução do algoritmo.
A expressão acima é para problemas de maximização.

Hill Climbing Estocástico

Influência do parâmetro T:

Exemplo (maximização):

$$\text{eval}(x_a)=35$$

$$\text{eval}(x_n)=45$$

$$P = \frac{1}{1 + e^{\frac{\text{eval}(x_a) - \text{eval}(x_n)}{T}}}$$

Se T muito alto → busca se torna aleatória

Se T muito baixo → busca se torna igual ao HC simples

Hill climbing simples

quanto maior T, menor a importância da diferença entre os pts competidores

T	p
1	1.00
5	0.88
10	0.73
20	0.62
50	0.55
10 ¹⁰	0.50..

Hill Climbing Estocástico

Exemplo:

$$\text{eval}(x_a)=35$$

$$T = 10$$

$$P = \frac{1}{1 + e^{\frac{\text{eval}(x_a) - \text{eval}(x_n)}{T}}}$$

eval(x_n)	dif	p
12	23	.09
25	10	.27
35	0	.50
40	-5	.62
55	-20	.88

Se $\text{eval}(x_n) = \text{eval}(x_a)$
A probabilidade de seleção é 0.5
Claro, não importa qual pt escolher

Se o novo pt é melhor, a probabilidade de seleção > 0.5.
Ela cresce com a diferença (negativa).

Simulated Annealing

Proposto por Kirkpatrick et al. (1983)

Inspirado pelo **processo de recozimento** de sistemas físicos (termodinâmica)

Determinado material é aquecido a altas temperaturas e depois resfriado lentamente, criando-se uma estrutura uniforme, produzindo um cristal.

Se o resfriamento for muito rápido não se produz uma estrutura perfeita

O parâmetro Temperatura (T) é adicionado de forma a escapar de ótimos locais (difere do Hill Climbing estocástico por T variar durante a execução)

Ele muda a probabilidade de se mover de um ponto a outro no espaço de busca.

No início T alto – similar a busca puramente aleatória

Nas ultimas iterações T muito baixo - similar HC simples

Simulated Annealing

Procedure
begin

$t = 0$

Initialize T

select a current string x_c at random

evaluate x_c

repeat

repeat

select a new point x_n in the neighborhood of x_c

if $eval(x_c) < eval(x_n)$ **then** $x_c = x_n$

Indica a probabilidade de x_n ser aceito

else if $random[0,1] <$

$$e^{\frac{eval(x_n) - eval(x_c)}{T}}$$

then $x_c = x_n$

until (termination-condition)

$T = g(T, t)$

$t = t + 1$

until (halting-criterion)

end

Sempre aceita novos pontos se melhores que atual

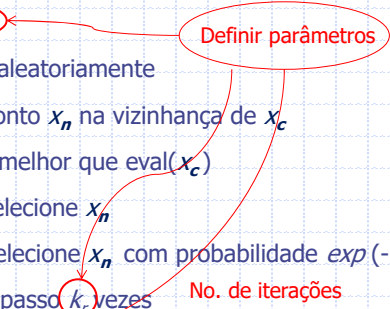
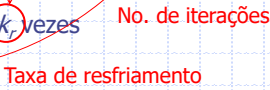
T inicia com valores altos (aquece) e vai diminuindo ao longo da busca (resfriamento)

No. fixo de iterações

Temperatura atingiu valor mín

Simulated Annealing

Maioria das implementações segue a sequência simples de passos:

- PASSO 1: $T = T_{\max}$ 
Selecione x_c aleatoriamente
- PASSO 2: Pegue um ponto x_n na vizinhança de x_c
Se $\text{eval}(x_n)$ melhor que $\text{eval}(x_c)$
então selecione x_n
senão selecione x_n com probabilidade $\exp(-\text{Deval}/T)$
Repita este passo k vezes 
- PASSO 3: Faça $T = r \cdot T$
Se $T > T_{\min}$ vá para o Passo 2
Senão vá para o Passo 1

Busca Tabu

Proposto por Glover, F. (1986):

Future paths for integer programming and links to artificial intelligence,
Computers & Operations Research, Vol. 13, pp . 533- 549 .

Uma Memória força o algoritmo a explorar novas áreas no espaço de busca.

- Admite soluções de piora para escapar de ótimos locais
- Não retorna a um ótimo local previamente visitado (condição desejada, mas não necessária)
- Cria uma **lista tabu** (LT): proíbe movimentos para soluções já visitadas
- Soluções proibidas por um certo número de iterações (prazo tabu)

Em geral, apenas alguns atributos das soluções já visitadas são tabu, em vez da solução completa (computacionalmente mais eficiente).

Busca Tabu

Algoritmo básico: $N(x)$ – vizinhança de x
 $\tilde{N}(x)$ – subconjunto admissível de $N(x)$ – (não tabu)

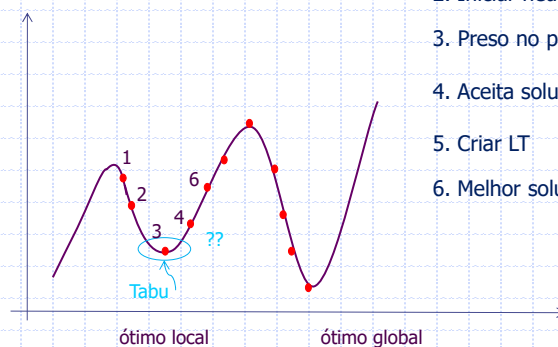
```

 $x \leftarrow x_0$ ;  $x_{\text{best}} \leftarrow x_0$ ;  $LT \leftarrow \emptyset$ 
while critério-de-parada do
  Obter a melhor solução  $x' \in \tilde{N}(x)$ 
  if  $\text{eval}(x') < \text{eval}(x_{\text{best}})$ 
  then  $x_{\text{best}} \leftarrow x'$ 
  else
    if  $\text{eval}(x') \geq \text{eval}(x)$ 
    then  $LT \leftarrow LT \cup \{x\}$ 
    end-if
   $x \leftarrow x'$ 
  Atualizar lista-tabu LT (prazos)
end-while
  
```

Então x é ótimo local
e vai para a LT

Busca Tabu

Exemplo: Problema de minimização



1. Selecionar um ponto
2. Iniciar heurística de descida
3. Preso no primeiro ótimo local
4. Aceita solução pior
5. Criar LT
6. Melhor solução vizinha

Observações

BT:

Memória adaptativa: guarda elementos importantes da historia da busca

- proibição de movimentos
- penalização de movimentos
- probabilidade de seleção de movimentos

Existem varias alternativas de implementações das LT.

SA e BT – criados com o objetivo de escapar do ótimo local

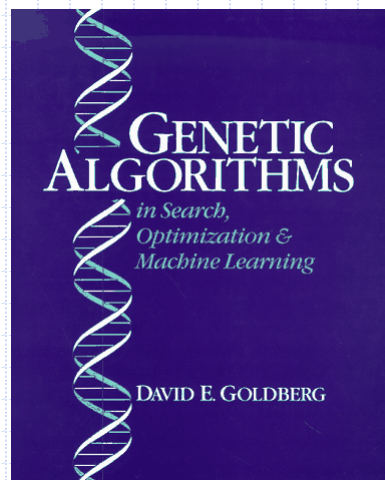
Bibliografia

**Genetic Algorithms in Search,
Optimization, and Machine
Learning**

David E. Goldberg

ISBN: 0201157675

Addison-Wesley Professional, 1989



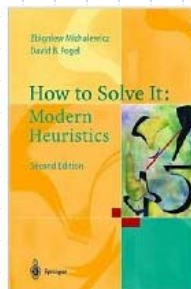
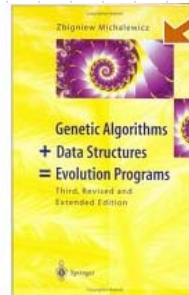
Bibliografia

Genetic Algorithms + Data Structures = Evolution Programs (Hardcover)

Zbigniew Michalewicz

ISBN: 3540606769

Springer, 3 edition (1998)



How to Solve It: Modern Heuristics

Zbigniew Michalewicz & David Fogel

ISBN: 3540224947

Springer, 2nd edition (2004)

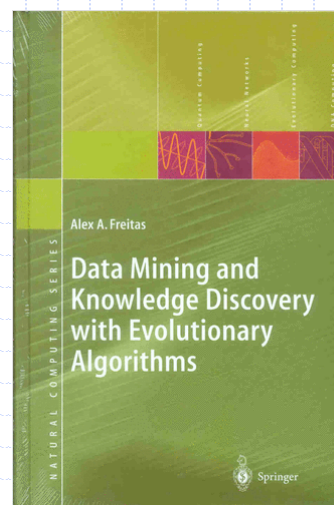
Bibliografia

Data Mining and Knowledge Discovery with Evolutionary Algorithms

Alex A. Freitas

ISBN: 3540433317

Springer; 1 edition (October 3, 2002)



Bibliografia

- ◆ HOLLAND, J.H. "Adaptation in Natural and Artificial Systems", University of Michigan Press, 1975.
- ◆ DAVIS, L. (ed.) "Handbook of Genetic Algorithms", Van Nostrand Reinhold, 1991.
- ◆ MITCHELL, M. "An Introduction to Genetic Algorithms", The MIT Press, 1996.
- ◆ BÄCK, T., FOGEL, D.B. & MICHALEWICZ, Z. (eds.) "Handbook of Evolutionary Computation", Institute of Physics Publishing and Oxford University Press, 1997.
- ◆ SCHWEFEL, H.-P. "Evolution and Optimum Seeking", Sixth-Generation Computer Technology Series, Wiley, 1995.

Outras informações

➤ Conferências principais:

- *IEEE International Conference on Evolutionary Computation*
- *Genetic and Evolutionary Computation Conference*

➤ Periódicos principais:

- *IEEE Transactions on Evolutionary Computation*
- *Evolutionary Computation (MIT Press)*

➤ Assinatura de periódicos:

<http://www.periodicos.capes.gov.br>

<http://ieeexplore.ieee.org/search/advsearch.jsp>